MGMTMSA 412 Capstone Project: StudyBuddy.AI

Team 10

UCLA

Presented by:
Rohan Ray, Josh Bastin,
Matt Balderrama,
Onkar Randad,
Arunava Singh

Date Presented:
October 30th, 2025

Our Project

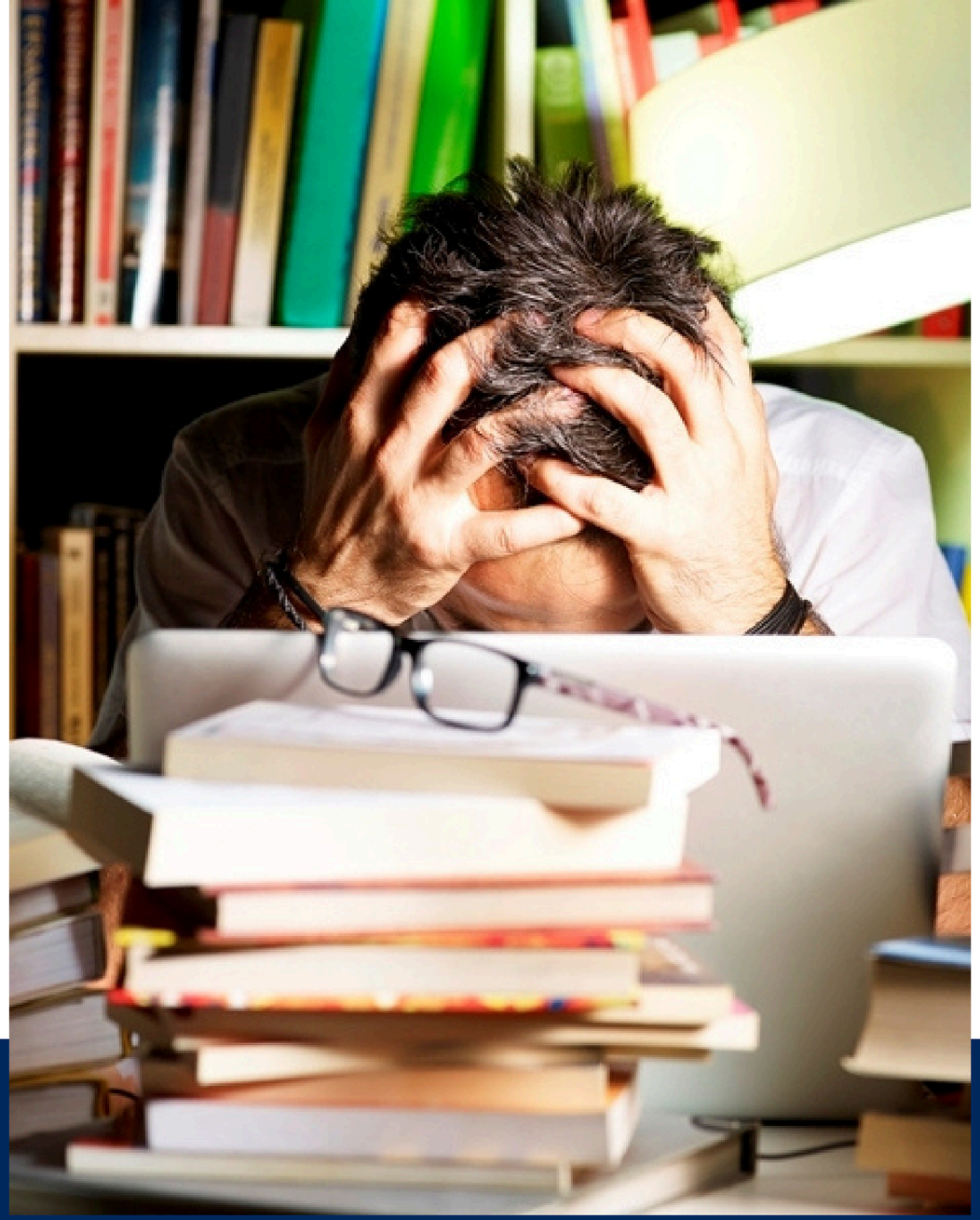
Our project focuses on **building an AI agent** using Traversaal AI's **Agent Pro framework** to explore how intelligent agents can solve real-world problems.

We adopt a **problem-first approach**, identifying a meaningful challenge and designing an agent that provides value to its stakeholders.

Problem Statement

Students struggle to study efficiently during **midterms and finals**, overwhelmed by scattered notes and materials that **lack personalized structure or focus**.

Our Goal: Create an AI-powered Study Buddy that transforms class content into organized, personalized, and interactive study guides, helping students learn smarter, not harder.



Proposed Solution

Our AI-powered Study Buddy Platform acts as a one-stop shop for all studying needs, helping students organize, learn, and track progress through intelligent automation

How it Solves the Problem

- **Personalized Study Guides:** Automatically generates summaries and planners based on uploaded class materials
- **Interactive Learning Tools:** Creates quizzes and flashcards from notes to reinforce key concepts
- **Smart Progress Dashboard:** Tracks goals, completion rates, and topic mastery through clear visual analytics
- **Actionable Insights:** Highlights weak areas, study time distribution, and suggested revisions for better performance

Back to Agenda

Our Journey

Our journey involved identifying the core learning problem, building two different StudyBuddy models with distinct architectures, and comparing their strengths and tradeoffs to understand which approach best supports personalized studying.

Identify student study challenges, explore Traversaal's tools, and define essential features to guide how users interact with StudyBuddy.

Stage 1: Problem Solving & Discovery

Create an initial version using a partially ReAct-style setup with modular tools, helping us explore early usability and workflow.

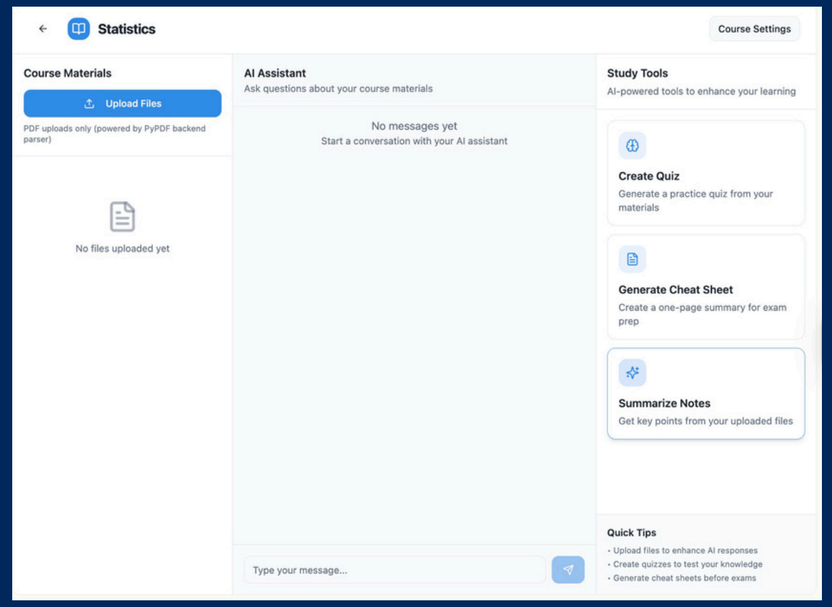
Stage 2: Build the First Model

Develop a second version with a streamlined orchestrator that unifies features, letting us compare speed, usability, and overall experience.

Stage 3: Build the Second Model

Version 1:
Router Version

The Frontend:



- Students land on a **personal dashboard** that shows total study time, average quiz score, streaks, and course cards.
- They **select a course** to open the course workspace, where all PDFs, notes, and practice problems are uploaded and organized.
- An **AI chat panel** lets students ask questions about their materials and get explanations, summaries, or guidance in real time.
- **Study Tools** on the right let them **generate quizzes, cheat sheets, and note summaries** directly from their uploaded files.
- As they study, **the dashboard updates analytics and progress charts**, helping them track performance and study habits over time.

The Backend:

- **FastAPI** receives each request and sends it to an orchestrator that **identifies the user's intent.**
- **Hybrid RAG retrieves** the most relevant content using **semantic search, BM25,** and **MMR filtering.**
- **Generator functions produce quizzes, summaries, study guides, or plans** based on the selected mode.
- **Structured outputs** with citations are returned to the **front end** for **display**.
- **SQLite stores user progress, mastery scores, and study history** for adaptive learning.

Version 1:

Demo

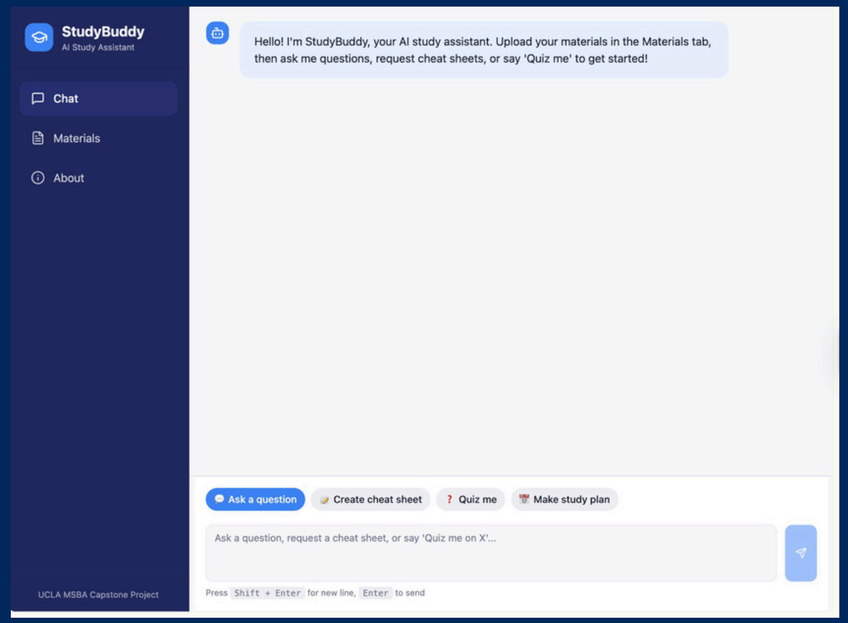
Version 1 Evaluation

	Excellent	Good	Satisfactory	Poor
Retrieval & Context Quality		✓		
Reasoning & Task Execution			✓	
Response Quality			✓	
Speed & Efficiency		✓		
Multi-Feature Usability	✓			

Version 1 delivers fast performance and strong multi-feature usability through its full dashboard experience, but its lighter reasoning and retrieval depth make it less capable on complex tasks compared to Version 2.

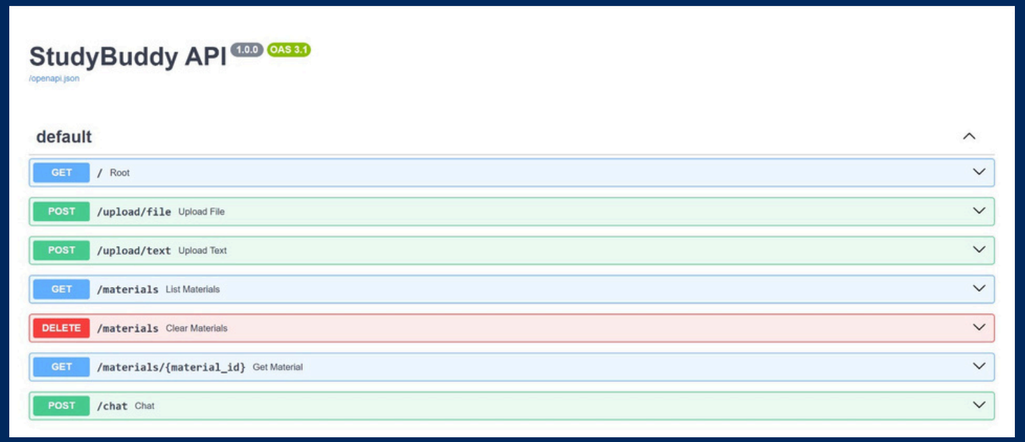
Version 2:
ReAct Version

The Frontend:



- Single **chat-based interface** where students upload files and ask questions i**n one place**.
- **Simple file upload panel for PDFs, slides, notes**, and text, which are immediately available to the agent.
- The **same chat box** is used for **Q&A, quiz requests, study plans, and cheat sheets**, so students don't have to learn different views or modes.
- **Responses** are **returned inline in the chat**, keeping the experience lightweight and focused on conversation rather than dashboards.

The Backend:



- **FastAPI** exposes endpoints for file upload and /chat, **passing requests** into a **StudyBuddyAgent**.
- Uploaded PDFs, docs, and slides are **parsed into text** and **stored** in an **in-memory material** store keyed by material IDs.
- The StudyBuddyAgent **wraps a ReactAgent that runs a ReAct loop**: it reasons about the user's query, selects tools, observes results, and iterates until it decides on a final answer.
- **Tools** include **listing** and **retrieving** materials, **searching** within them, and **generating** quizzes, study plans, and cheat sheets.
- Each chat call **combines LLM reasoning + tool calls** to return a structured response (answer, quiz, plan, or cheat sheet) back to the front end.

Version 2:

Demo

Version 2 Evaluation

	Excellent	Good	Satisfactory	Poor
Retrieval & Context Quality		✓		
Reasoning & Task Execution	✓			
Response Quality	✓			
Speed & Efficiency			✓	
Multi-Feature Usability			✓	

Version 2 delivers far stronger reasoning and richer, more tailored responses through its ReAct loop, but its slower speed and simpler single-chat interface make it less feature-complete and less efficient than Version 1.

Comparing the Versions

Usability & Interface Depth

Version 1 provides a fuller, feature-rich interface with dashboards and course tools, while Version 2 simplifies everything into a single chat, offering ease of use but fewer built-in features.

Reasoning Strength & Output Quality

Version 2 delivers stronger reasoning and richer responses thanks to its ReAct architecture, whereas Version 1 is more consistent but less deep on complex or multi-step tasks.

Performance, Speed & Scalability

Version 1 is faster and more efficient, making it highly scalable, while Version 2 is slower due to its reasoning loopsbut more flexible and context-aware.

Final Thoughts: By combining Version 1's intuitive dashboard with Version 2's advanced ReAct-powered reasoning, we can build a unified platform that offers both ease of use and deeper, more intelligent learning support.

What we have Learned

Problem-First Still Wins

Designing around the student's pain points led to clearer features, stronger usability, and more purposeful agent behavior.

Two Models Reveal Tradeoffs

Building both versions showed how dashboard-driven usability (V1) and advanced reasoning depth (V2) serve different but complementary needs.

RAG + ReAct Creates Real Intelligence

Combining retrieval with step-by-step reasoning significantly improves accuracy, context-awareness, and adaptive explanations.

Usability Is as Important as Intelligence

Even the most capable AI requires a clean, intuitive interface. Students adopt tools that feel simple, stable, and easy to navigate.

UCLA

Thank You

Team 10